TECHNICAL DOCUMENTATION & ARCHITECTURE MANUAL

Japanese Flashcards — The Ultimate Beginner's Guide & Mentor Manual

Written by Antigravity — Your AI Pair-Programming Mentor

---

0. A Welcome Note From Your Mentor

Hello! I am your AI Coding Guide and Pair-Programming Mentor.

When you first start coding, looking at hundreds of lines of HTML, CSS, and JavaScript can feel overwhelming. You might see terms like `onSnapshot`, `Set`, `async/await`, `AudioContext`, or `Fisher-Yates` and feel like it's a foreign language.

Here is my core advice to you as your mentor:

> Every expert developer was once a beginner who felt confused. Coding is not about memorizing syntax — it is about learning how to break big problems down into small, logical steps.

This manual is structured as your personal mentorship playbook. It explains how this entire Japanese Flashcard application was designed, how data flows, why specific tools were chosen, how real bugs were diagnosed and solved, and how you can think like a professional software engineer.

---

1. Project Overview & Core Mission

What is this project?

Japanese Flashcards is a single-page web application designed for learning Japanese alphabets (**Hiragana**, **Katakana**), **Numbers**, and foundational **JLPT N5 Kanji**.

Core Features & Design Philosophy:

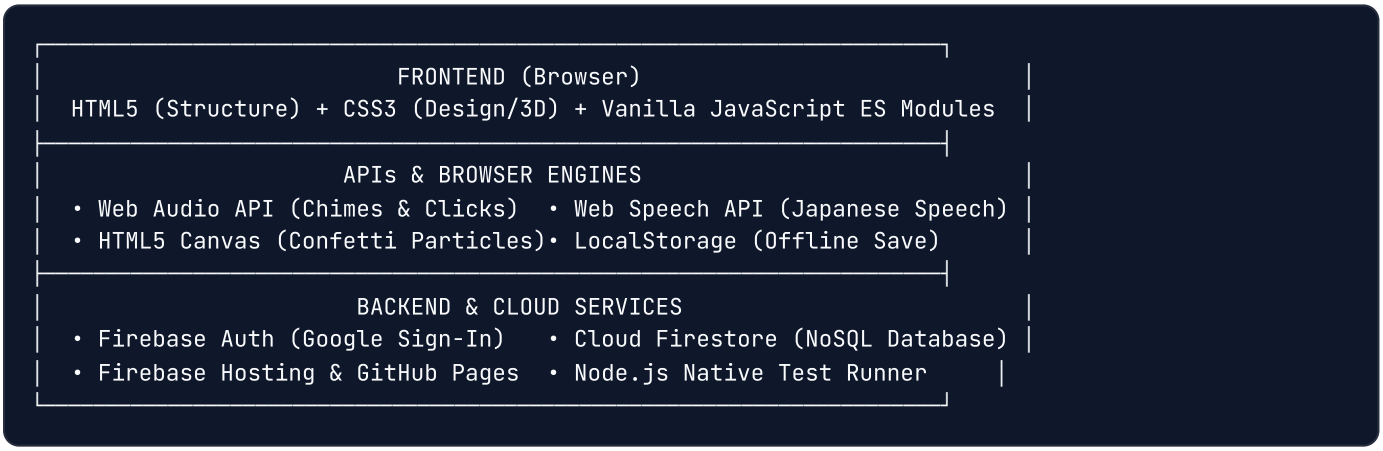
- **Zero-Build Architecture**: Built entirely with plain HTML, CSS, and Vanilla JavaScript inside a single file (`index.html`). Requires no build tools (like Webpack, Vite, or Babel), making the app load instantly and making the code easy for a beginner to inspect and modify.
- **5 Study Decks**:
- **Hiragana** (46 characters) — Standard Japanese phonetic script (Red theme).
- **Katakana** (46 characters) — Script used for foreign loanwords (Blue theme).
- **Both** (92 characters) — Combined Hiragana + Katakana concatenated deck (Purple theme).
- **Numbers** (12 characters) — Japanese numbers (1–9, 10, 100, 1000) with numeral badges (Orange theme).

- **Kanji** (42 characters) — Foundational JLPT N5 ideograms with readings, meanings, and visual emoji mnemonics (Emerald Green theme).
- **Two Distinct Study Modes**:
- **Learn Mode**: Standard flashcard view with card flip, romaji readings, Japanese speech pronunciation audio, and category hint tags.
- **Revision Mode**: Distraction-free Midnight Slate theme (`#0f172a` gradient) presenting an interactive **3-choice multiple-choice quiz** on card flip, live streak counters (`🔥 X streak`), keyboard shortcuts (`1`, `2`, `3`), and celebratory fireworks.
- **Cross-Device Progress Synchronization**: Masters known cards across sessions via local browser storage (`localStorage`) and syncs in real-time across mobile/desktop devices using Google Cloud Firebase.
- **Visual Progress Matrix**: Interactive **Character Grid modal** displaying all deck characters in Gojūon layout with live filter pills (`All`, `Mastered ✓`, `Unmastered`).
- **Sound & FX Engine**: Zero-file Web Audio synthesizer for tactile card clicks and major-third success chimes, plus HTML5 Canvas particle fireworks for deck mastery.

---

2. Tech Stack Explained for Beginners

In software development, a **Tech Stack** refers to the collection of tools, libraries, and programming languages used to build an application. Here is what we used and why:



1. HTML5 (HyperText Markup Language)

- **What it is**: The skeleton of any website. It defines structural tags like buttons, headings, text cards, dialog overlays, and canvases.
- **Key Elements Used**:
- `<div class="card-scene">`: Container wrapping the 3D flashcard.
- `<button>`: Clickable interactive elements for deck tabs, mode switches, and quiz options.
- `<canvas id="confetti-canvas">`: Drawing surface for rendering particle fireworks.
- `<meta name="viewport">`: Ensures the webpage scales correctly on mobile phones.

2. CSS3 (Cascading Style Sheets)

- **What it is**: The design, colors, typography, layout alignments, and animations of the app.
- **Key Techniques Used**:
- **Flexbox (`display: flex`) & Grid (`display: grid`)**: Alignment systems used to center flashcards, position navigation tabs, and lay out the 5-column Gojūon character matrix.
- **3D Perspective & Transforms**: CSS properties (`transform-style: preserve-3d`, `rotateY(180deg)`, `backface-visibility: hidden`) that create realistic 3D card flips.
- **CSS Custom Properties & Dark Theme**: `data-theme="dark"` styling that dynamically adjusts page gradients, card contrast, and modal glassmorphism blur filters (`backdrop-filter: blur(8px)`).

3. Vanilla JavaScript (ES Modules)

- **What it is**: The brain of the website. JavaScript controls interactive state, button clicks, keyboard shortcuts, quiz distractor generation, and cloud sync.
- **"Vanilla" JS**: Standard JavaScript written without heavy third-party frameworks (like React, Vue, or Angular). Keeps the app ultra-fast, zero-dependency, and lightweight.
- **ES Modules (`<script type="module">`)**: A modern browser standard allowing us to import Firebase tools directly from Google's CDN URLs (`https://www.gstatic.com/...`) using native `import` statements.

4. Cloud Firebase & Cloud Firestore

- **Firebase Auth**: Manages user authentication securely, allowing users to sign in with Google (`signInWithPopup` with fallback to `signInWithRedirect`).
- **Cloud Firestore**: A NoSQL real-time cloud database where each signed-in user's mastered cards are stored at `users/{uid}/progress/known`. Changes on one device trigger live snapshot updates (`onSnapshot`) on all other signed-in devices.

5. Native Browser APIs (Zero External Download Files!)

- **Web Audio API**: Programmatically synthesizes sound waves in JavaScript (sine waves for card click tones, triangle waves for major-third success chimes, sawtooth waves for error tones). Requires zero `.mp3` files!
- **Web Speech API (`speechSynthesis`)**: Uses the browser's native speech voice engine set to Japanese (`lang: "ja-JP"`) to read characters aloud when clicking the speaker button or pressing `S`.
- **HTML5 Canvas Context (`getContext('2d')`)**: A 2D rendering context used to animate particle gravity acceleration, velocity vectors, rotation, and alpha fade for 75 confetti pieces.
- **LocalStorage API**: Browser key-value storage (`localStorage.getItem()`, `setItem()`) that saves user progress (`kana-known-cards`), theme preference (`kana-theme`), and sound toggle state (`kana-sound`) offline.

6. Node.js Native Test Runner (`node --test`)

- **What it is**: A built-in testing framework included in Node.js. Runs our automated test suite (`tests/app.test.mjs`), executing 30 unit tests in under 150 milliseconds to verify code correctness.

---

3. Core Programming Concepts, Logic & Methodologies

A. State & State Management

In programming, **State** refers to the current snapshot of data in your app at any given moment.

In our app, state includes:

```
let script = "hiragana";      // Which deck tab is active? ("hiragana" | "katakana" | "numbers" |
"kanji" | "both")
let deck = [...H];             // Array of card objects currently displayed
let idx = 0;                   // Index position of the current card (0 to deck.length - 1)
let flipped = false;           // Is the card showing front (false) or back (true)?
let shuffled = false;          // Is the deck in original order (false) or randomized (true)?
let known = new Set();         // Set of character strings marked as known by the user
let appMode = "learn";         // Current study mode ("learn" | "revision")
let revisionStreak = 0;        // Consecutive correct quiz answers in Revision Mode
```

Whenever any state variable changes, a single central function called `render()` runs to update all visual text, checkmarks, progress bars, and themes on screen.

B. Data Structures: `Set` vs. `Array`

Why did we use a `Set` for `known` cards instead of an `Array` ?

```
// Array representation (ORDERED LIST, DUPLICATES ALLOWED)
const knownArray = ["あ", "い", "あ"]; // Duplicate "あ" exists!

// Set representation (UNIQUE VALUES ONLY, INSTANT HASH LOOKUP)
const knownSet = new Set(["あ", "い"]); // Duplicate "あ" is automatically rejected
```

Operation	Array (`[]`)	Set (`new Set()`)	Why Set wins here
Adding an item	`arr.push("あ")` (can create duplicate entries)	`set.add("あ")` (automatically rejects duplicates)	Guarantees clean list of unique character strings
Checking if known	`arr.includes("あ")` (scans items one-by-one: $O(N)$ speed)	`set.has("あ")` (instant hash lookup: $O(1)$ speed)	Ultra-fast check on every single card render
Deleting an item	`arr.splice(index, 1)` (requires searching index first)	`set.delete("あ")` (instant removal)	Clean single-line toggle when unmarking cards

C. Character-Based Tracking vs. Index-Based Tracking

Never track user progress by array index (`0, 1, 2`) when items can be re-ordered or shuffled!

```
❌ WRONG (Index-based):
   known = [0, 2] → User knows card at index 0 ("あ") and index 2 ("う").
   User clicks "Shuffle"!
   Now index 0 is "ん" and index 2 is "き".
   Result: The app mistakenly marks "ん" and "き" as known!

☑ CORRECT (Character-based):
   known = Set(["あ", "う"]) → User knows character "あ" and character "う".
   User clicks "Shuffle"!
   Card "あ" moves to index 14.
   Result: `known.has(card.c)` checks card.c ("あ") → returns TRUE! Progress stays 100%
accurate.
```

D. Essential JavaScript Building Blocks Explained

For a beginner learning JavaScript, here are the exact building blocks used throughout our application:

1. Variables (`let` vs `const`)

- `const` : Used for values that will never be reassigned (e.g., `const H = [...]` , `const markBtn = ...`).
- `let` : Used for variables whose values change over time (e.g., `let idx = 0` , `let flipped = false`).

2. JavaScript Array Iteration Methods

- `forEach` : Loops through every item in an array to perform an action. `javascript deck.forEach((card, index) ⇒ { console.log(card.c); });`
- `filter` : Creates a new array containing only items that match a true condition. `javascript const knownInDeck = deck.filter(c ⇒ known.has(c.c)).length;`
- `map` : Transforms every item in an array into a new format. `javascript const options = distractors.map(text ⇒ ({ text: text, isCorrect: false }));`
- `some` : Returns `true` if at least one item in the array satisfies a condition. `javascript const isHiragana = H.some(card ⇒ card.c ⩵ ch);`

3. DOM Manipulation Methods

- `document.getElementById("mark-btn")` : Finds an HTML element by its `id` .
- `element.textContent = "Hello"` : Changes the visible text inside an element.
- `element.classList.toggle("flipped", flipped)` : Adds or removes a CSS class automatically based on a boolean value.
- `element.style.display = "flex"` : Directly updates an element's inline CSS styling.

4. The Ternary Operator (`condition ? valueIfTrue : valueIfFalse`)

A shorthand way to write an `if / else` statement in a single line:

```
markBtn.textContent = known.has(card.c) ? "✓ Known" : "Mark Known";
```

5. Short-Circuit Logical Operators (`&&` and `||`)

- `||` **(Default Fallback Operator)**: Returns the left value if it exists, otherwise uses the right default. `javascript let soundEnabled = (localStorage.getItem("kana-sound") || "on") === "on";`
- `&&` **(Conditional Guard Operator)**: Executes the right side only if the left side is true. `javascript if (currentUser) saveKnown(); // Saves to cloud only if user is logged in`

6. Synchronous vs. Asynchronous Code (`async / await`)

- **Synchronous**: Executes line-by-line instantly (e.g. flipping a card).
- **Asynchronous**: Tasks that take time (like downloading Firestore database records). Using `async` and `await` ensures the browser doesn't freeze while waiting for network responses: `javascript async function saveKnown() { saveLocalKnown(); // Instant local save if (!currentUser) return; await setDoc(userDocRef(currentUser.uid), { cards: [...known] }); // Async cloud write }`

7. Event Delegation & `closest()`

When listening for user clicks on a container, `e.target.closest(".speaker-btn")` checks if the clicked element (or any of its parent elements) matches the `.speaker-btn` class:

```
cardScene.addEventListener("click", e => {
  if (e.target.closest(".speaker-btn")) return;
  flipped = !flipped;
  render();
});
```

E. How an Engineer Thinks: Mental Models & Implementation Methodologies

Before writing a single line of code, software engineers use **Mental Models** and **Design Methodologies** to plan how the system should work. Here are the exact mental models used to design this app:

1. Single Source of Truth (SSOT)

- **The Problem**: If 5 different places in code track whether "あ" is known (the card object, a variable, DOM attributes, storage, database), they will quickly get out of sync.
- **The Solution**: Maintain **one single canonical variable** (`known = new Set()`). All UI elements (checkmarks, progress bars, mastery badges, grid tiles) derive their state by querying this single set (`known.has(card.c)`).

2. Optimistic UI Updates vs. Pessimistic UI Updates

- **Pessimistic UI**: Waiting 1–2 seconds for the database server to reply before showing the green checkmark to the user. Feels slow and laggy.
- **Optimistic UI (Our Approach)**: Update the local `known` set, save to `localStorage`, and trigger pop animations instantly (0ms latency). Then send the cloud `setDoc()` request asynchronously in the background.

3. Separation of Concerns (SoC)

Divide the application into distinct, isolated layers: - **Data Layer**: Clean static arrays (`H` , `K` , `N` , `KJ`). - **Pure Helper Functions**: Inputs $\to$ Outputs with zero side-effects (`shuffleArr()` , `generateQuizOptions()` , `getDeckTitle()`). - **State Mutators**: Event listeners that change data variables (`markBtn` listener, `goTo()` , `selectQuizOption()`). - **Render Engine**: Pure UI sync loop (`render()` , `renderGrid()` , `renderQuizUI()`). - **Side-Effect Drivers**: Web Audio API (`playFX()`), Web Speech API (`speak()`), Canvas (`animateConfetti()`).

4. Lazy Evaluation & Deferred Resource Allocation

Don't allocate memory or run heavy calculations until the user actually needs them: - **Audio Context**: `audioCtx` is initialized lazily on the first user click (`playFX()`), satisfying browser autoplay policies. - **Quiz Generation**: `currentQuiz` is generated on demand when the user flips a card in Revision mode, rather than pre-building quiz objects for all 146 cards on startup.

5. Concurrency & Reflow Guarding

- In `goTo()` , an `animating = true` boolean lock prevents rapid multi-clicking from corrupting CSS card transitions.
- In `celebrateKnown()` , `void markBtn.offsetWidth` forces a single browser reflow so CSS keyframe animations re-trigger cleanly on consecutive clicks.

6. Cross-Platform Boundary Thinking

Always ask: *"What if this runs on an iPhone? What if popups are blocked? What if Wi-Fi drops?"* - **Auth Boundary**: If mobile browsers block `signInWithPopup()` , automatically fall back to `signInWithRedirect()` . - **Audio Boundary**: Wrap sound functions in `try/catch` so missing Web Audio support never crashes the app. - **Network Boundary**: Offline progress saves to `localStorage` and automatically union-merges with Firestore upon reconnecting.

---

4. Decks & Data Architecture

Each Japanese flashcard is represented as a clean JavaScript object inside structured data arrays:

Data Schemas:

```
// Kana Card Schema: { c: character, r: romaji, g: group/family }
{ c: "あ", r: "a", g: "Vowels" }

// Number Card Schema: { c: kanji, r: romaji, g: group, n: Arabic numeral }
{ c: "一", r: "ichi", g: "Ones", n: "1" }

// Kanji Card Schema: { c: kanji, r: readings & English meaning, g: concept group }
{ c: "日", r: "nichi, hi · sun/day", g: "Nature" }
```

Visual Kanji Emoji Mapping:

To make learning Kanji intuitive, we map each Kanji character to a representative visual emoji icon:

```
const KANJI_EMOJIS = {
  "日":"☀️", "月":"🌙", "木":"🌳", "水":"💧", "火":"🔥", "土":"⛰️", "金":"🪙",
  "山":"⛰️", "川":"🌊", "人":"🧍", "女":"👩", "男":"👨", "子":"👶", "目":"👁️"
};
```

Color Palette Identifiers:

Each script deck has its own dedicated light and dark theme color identity:

```
const COLORS_LIGHT = {
  hiragana: { bg: "#c0392b", light: "#e74c3c", accent: "#c0392b" }, // Crimson Red
  katakana: { bg: "#1a5276", light: "#2980b9", accent: "#1a5276" }, // Ocean Blue
  both:     { bg: "#6c3483", light: "#8e44ad", accent: "#6c3483" }, // Royal Purple
  numbers:  { bg: "#d35400", light: "#e67e22", accent: "#d35400" }, // Vibrant Orange
  kanji:    { bg: "#117864", light: "#16a085", accent: "#117864" }  // Emerald Green
};
```

---

5. Major Bugs Found and How They Were Solved

Real-world coding is 30% writing new features and 70% discovering and solving subtle bugs. Here are the top bugs solved in this codebase:

Bug Symptom	Root Cause & Fix
1. Mobile tap-to-flip flipped card twice instantly.	Mobile browsers fire a synthetic `click` event immediately after `touchend`. Fixed by setting a `touchHandled` flag + calling `preventDefault()`.
2. Resetting deck wiped ALL user progress across all 5 decks!	`resetConfirmBtn` executed `known = new Set()`, wiping Katakana/Kanji when resetting Hiragana. Fixed by using selective deletion: `deck.forEach(c ⇒ known.delete(c.c))`.
3. Mastering "Both" deck toasted "Hiragana Mastered!".	Toast called `scriptLabel(deck[0].c)` which returned "Hiragana" because deck[0] is Hiragana. Fixed by using `getDeckTitle()` which correctly returns "Both".
4. Unmarking a card blocked future re-mastery celebrations.	`masteredDecks` set retained deck name even when unmastered. Fixed by executing `masteredDecks.delete()` whenever `isMastered` becomes false.
5. Desktop 3D tilt blocked card flip animation.	Mousemove listener set inline CSS `style.transform`, blocking CSS `.flipped`. Fixed by resetting inline transform in `render()` and ignoring tilt when flipped.
6. Background hotkeys fired behind open popup dialogs.	Pressing Space/Arrows flipped cards behind open dialog popups. Fixed by adding `isModalOpen` guard check.

---

6. Automated Testing: Why & How We Test

What is Automated Testing?

Instead of manually opening the web browser and clicking buttons for 20 minutes to verify every feature works after making a code change, we write a **Test Script** (`tests/app.test.mjs`) that automatically checks 30 key scenarios in milliseconds!

Test Assertions (`assert.equal` , `assert.ok` , `assert.deepEqual`):

In automated testing, an **Assertion** checks if the actual code result matches our expected result:

```
// Test if Hiragana deck length is exactly 46
assert.equal(env.H.length, 46);

// Test if 'あ' is marked as known
assert.ok(known.has('あ'), 'Card あ should be marked known');

// Test if two arrays match exactly
assert.deepEqual(restoredArray, originalArray);
```

Running the Test Suite:

```
node --test tests/app.test.mjs
```

Sample Output:

```
▶ Japanese Flashcards Unit & Integration Tests
  ✓ 1. Card Decks Data Integrity (5 tests)
  ✓ 2. Quiz Distractor & Option Generator (1 test)
  ✓ 3. Deck Building & Shuffling (2 tests)
  ✓ 4. Deck Title & Label Formatting (2 tests)
  ✓ 5. Theme Colors Configuration (1 test)
  ✓ 6. Selective Deck Reset Behavior (1 test)
  ✓ 7. Mastery Calculation & Unmastering (1 test)
  ✓ 8. Revision Streak Engine (1 test)
  ✓ 9. Character Grid Filtering & Empty States (1 test)
  ✓ 10. LocalStorage Synchronization (1 test)
  ✓ 11. Multi-Device & Account Mail Sync Scenarios (7 tests)
  ✓ 12. Known Cards Sync & State Consistency on Shuffle Cases (7 tests)
✓ Japanese Flashcards Unit & Integration Tests (30 passing, 0 failing)
```

---

7. Key Takeaways & Best Practices for Beginners

1. **Keep it simple before adding frameworks**: You don't always need React, Vue, or Vite to build modern, real-time web apps. Plain HTML, CSS, and Vanilla JavaScript can take you extremely far.
2. **Never track state by position or index when list items can re-order**: Always track items by a unique identifier (like character string `card.c` or a unique ID `card.id`).

3. **Use the right data structures**: Use `Set` for unique lookup collections ($O(1)$ speed) and `Array` for ordered sequences.
4. **Break code into small helper functions**: Keep functions small and focused on a single job (`render()`, `saveKnown()`, `playFX()`, `showToast()`).
5. **Build defensive UI guards**: Always check if modals are open or if text inputs are focused before handling global keyboard shortcuts.
6. **Embrace debugging & testing as normal parts of coding**: Bugs are not failures — they are how you learn how browsers, events, and databases work!

---

8. Guide's Troubleshooting & DevTools Debugging Playbook

When your code doesn't work as expected, **don't guess blindly**. As your mentor, here is the exact 5-step debugging playbook I use:

```
Step 1: Open F12 DevTools ──► Step 2: Read Console Logs ──► Step 3: Check Network Tab ──► Step 4: Inspect LocalStorage
```

Step 1: Open Browser Developer Tools

Press `F12` (or `Right-Click → Inspect`) in Chrome, Edge, or Firefox.

Step 2: The Console Tab (`console.log`)

Look for red error messages. JavaScript errors display the exact file name and line number where the problem occurred (e.g. `index.html:1150`).

Step 3: The Application / Storage Tab

Click **Application → Local Storage → http://localhost** (or your live site domain). You can inspect `kana-known-cards` directly to see what data is stored on your device!

Step 4: The Network Tab

If cloud sign-in or Firestore sync fails, open the **Network Tab** to inspect HTTP requests. Look for red `403 Forbidden` or `400 Bad Request` status codes.

8B. Important Error Messages & Log Solving Playbook

Here is a practical reference guide covering the 7 most common web development errors, how to read their log output, and how to solve them:

1. `Uncaught TypeError: Cannot read properties of undefined (reading 'xyz')`

- **Cause**: Attempting to access property `.xyz` on a variable that evaluates to `undefined` or `null`.
- **Diagnostic Step**: Check `console.log("Variable value:", obj)` right before the error line.

- **Solution**: Use **Optional Chaining (`?.`)** or **Nullish Coalescing (`??`)**: `javascript // Safe optional chaining: const name = user?.displayName ?? "Guest User";`

2. `Firebase Error: auth/requests-from-referer-custom-domain-denied` **(HTTP 403)**

- **Cause**: Google Cloud Console API Key restrictions block requests from a new hosting domain origin.
- **Diagnostic Step**: Check DevTools Network tab → filter by `identitytoolkit.googleapis.com` → view Response JSON.
- **Solution**: Open Google Cloud Console → Credentials → API Key → Add `https://your-domain.web.app/*` to allowed HTTP referrers.

3. `FirebaseError: Missing or insufficient permissions`

- **Cause**: Cloud Firestore Security Rules (`firestore.rules`) rejected the read/write request because user is unauthenticated or `request.auth.uid ≠ userId`.
- **Diagnostic Step**: Run `console.log("Current Auth User:", auth.currentUser)`.
- **Solution**: Verify the user signs in before `setDoc()` executes, and check `firestore.rules` for rule matching logic.

4. `NotAllowedError: play() failed because the user didn't interact with the document first`

- **Cause**: Web browser autoplay security policies block AudioContext sound playback before the user's first click.
- **Diagnostic Step**: Check `audioCtx.state`. If state is `"suspended"`, calling `.start()` throws an exception.
- **Solution**: Initialize or resume AudioContext inside a user click handler: `javascript if (audioCtx.state === "suspended") audioCtx.resume();`

5. `Uncaught ReferenceError: variableName is not defined`

- **Cause**: Calling a variable before declaring it, misspelling variable name, or variable is scoped inside another function block.
- **Diagnostic Step**: Check line number in DevTools Console and inspect scope declarations.
- **Solution**: Declare variable using `let` or `const` at top-level scope if accessed across multiple functions.

6. `QuotaExceededError: Failed to execute 'setItem' on 'Storage'`

- **Cause**: Browser `localStorage` storage quota (typically 5MB) was reached.
- **Diagnostic Step**: Check DevTools Application tab → Local Storage → inspect total byte sizes.
- **Solution**: Wrap `localStorage.setItem()` in a `try/catch` block and clear obsolete key entries.

7. CSS Layout Bug: Inline Style Overrides CSS Classes

- **Cause**: Setting `element.style.display = "block"` via inline JavaScript permanently overrides responsive CSS classes like `display: flex`.

- **Diagnostic Step**: Right-click element → Inspect Element → look at Styles pane to see crossed-out rules.
- **Solution**: Avoid hardcoded inline style overrides; use `element.classList.toggle()` to control CSS rules cleanly.

---

9. Advanced Web Engineering Patterns Used

1. Fluid Typography Math (`clamp()`)

Instead of writing 5 different CSS media queries for different phone screen sizes, we use CSS `clamp(min, preferred, max)`:

```
/* Font size automatically scales smoothly between 1.4rem and 2rem based on screen width (5vw) */
h1 { font-size: clamp(1.4rem, 5vw, 2rem); }
```

2. Speech Cancellation (`speechSynthesis.cancel()`)

When users rapidly press key `S` or click the speaker button multiple times, audio utterances would queue up and play sequentially for 10 seconds.

To solve this, we execute `speechSynthesis.cancel()` before starting a new utterance:

```
function speak(text) {
  if (!("speechSynthesis" in window)) return;
  speechSynthesis.cancel(); // Instantly stops any speech currently playing!
  const u = new SpeechSynthesisUtterance(text);
  u.lang = "ja-JP";
  u.rate = 0.8;
  speechSynthesis.speak(u);
}
```

---

10. Your Roadmap for Future Learning

Congratulations on studying this application! Now that you understand how this single-page web app works, here is your recommended mentor learning path:

1. **Master Core JavaScript Basics**: Keep practicing arrays, objects, functions, loops, and ES6 arrow syntax (`() => {}`).
2. **Learn Modern Frameworks (React or Streamlit)**: Once comfortable with Vanilla JS, learn React components (`useState` , `useEffect`) for larger data applications.
3. **Explore Backend APIs (Node.js / Express or Python FastAPI)**: Learn how to build your own REST API servers to save data to database tables.
4. **PWA (Progressive Web Apps)**: Learn how to add a `manifest.json` and `service-worker.js` so users can install your flashcards app onto their phone home screen offline!